

YOLO + Livox 인지 파이프라인 정밀 분석

`perception_ws` 의 세 패키지 — `detect_msgs` , `livox_clustering` , `livox_camera_fusion` — 를 코드 라인 단위로 분해해서 정리한 문서입니다.

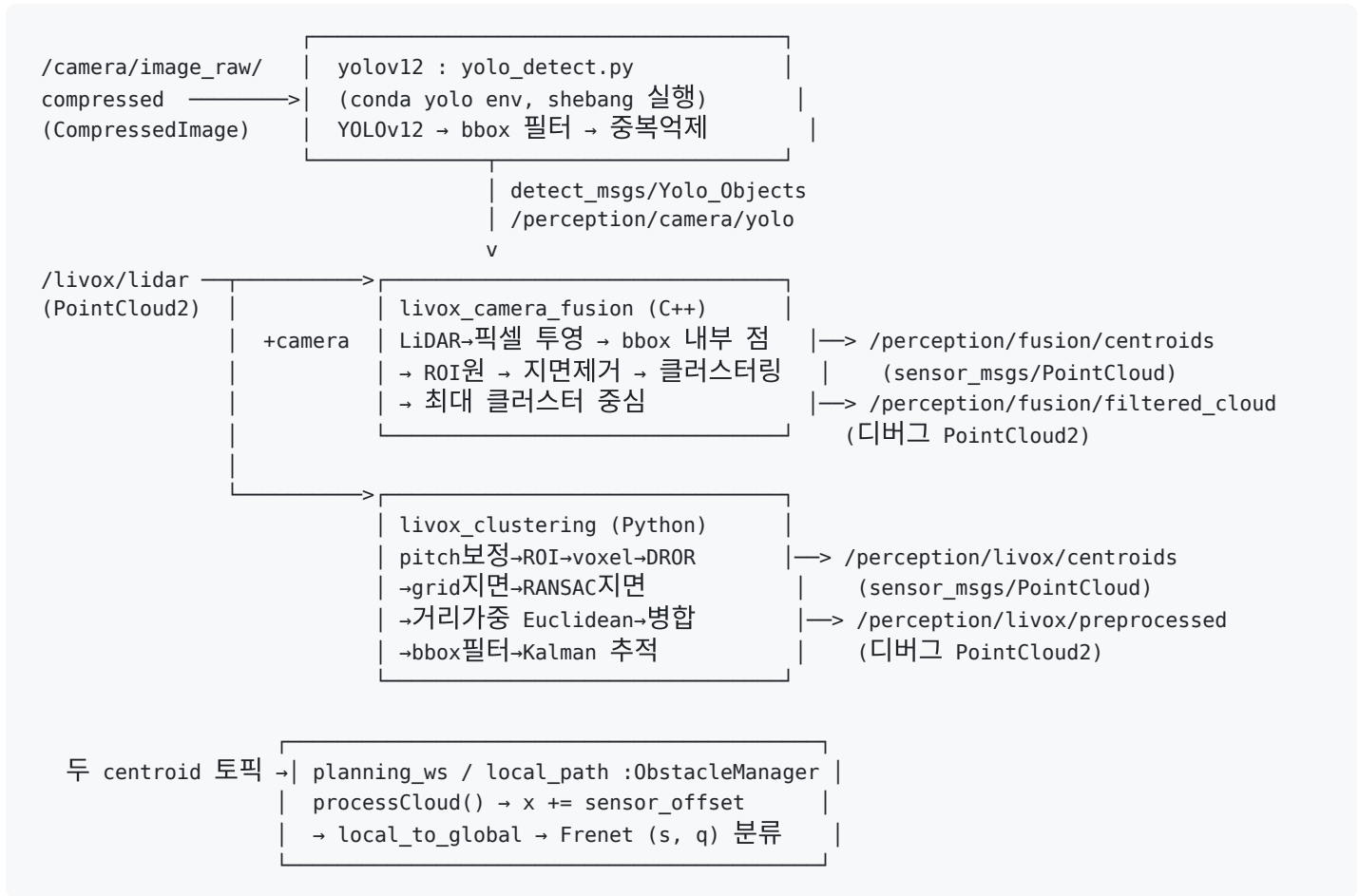
- 대상 브랜치: `2026_0chang_2nd` (`a0b63dab` `ochang_2nd_code`)
- ROS: Noetic / catkin / C++14 / Python3

목차

- 0. 전체 그림
- 1. `detect_msgs`
- 2. `livox_clustering`
- 3. `livox_camera_fusion`
- 4. 두 파이프라인 비교
- 5. 죽은 코드 / 버그 총정리
- 6. 우선순위별 개선 제안
- 7. 디버깅 치트시트

0. 전체 그림

0.1 노드/토픽 배선도



- 두 파이프라인은 **완전히 독립**입니다. 서로 토픽을 주고받지 않고, planning 쪽 `obstacle_manager.cpp:46-47` 에서 각각 `jagyeong_sub_ / minjae_sub_` 로 따로 구독해 합칩니다. 즉 **한쪽이 죽어도 다른 쪽은 계속 동작**하는 이중화 구조입니다.
- 두 노드 모두 최종 산출물은 **"2D 점 리스트"** 하나뿐입니다. `sensor_msgs/PointCloud` 의 `points[i] = (x, y, 0)` — 클래스/크기/속도 정보는 전부 버려집니다. planning 은 이걸 "장애물 중심점"으로만 씁니다.

0.2 실행 방법

```
roslaunch perception_bringup perception.launch # fusion + livox_clustering (2개 노드)
rosrun yolov12 yolo_detect.py                  # conda yolo env (shebang)
```

`perception_bringup/launch/perception.launch` 는 두 노드의 launch 를 `<include>` 하면서 토픽 이름만 넘겨줍니다. `filtered_cloud_topic / preprocessed_topic / frame_name / frame_id` 는 넘기지 않아서 각 launch 의 기본값이 쓰입니다.

1. detect_msgs

1.1 역할

카메라 인지 결과(YOLO bbox)를 다른 노드로 전달하기 위한 **메시지 정의 전용 패키지**. 실행 노드가 없고, 빌드하면 C++ 헤더 (`detect_msgs/Yolo_Objects.h`)와 Python 모듈 (`detect_msgs.msg`)만 생성됩니다.

1.2 파일 구조

```
detect_msgs/
├── CMakeLists.txt
├── package.xml
└── msg/
    ├── Objects.msg      ← 실제 사용
    ├── Yolo_Objects.msg ← 실제 사용
    ├── detected_object.msg ← 죽은 코드
    └── detected_array.msg ← 죽은 코드
```

1.3 메시지 정의 상세

Objects.msg — 검출 객체 1개

```
int32 Class # YOLO 클래스 id : 0=ERP-42, 1=drum, 2=cone
int32 id    # 프레임 내 인덱스 (0,1,2,...). 프레임 간 추적 id 아님!
int32 x1    # bbox 좌상단 x (픽셀)
int32 x2    # bbox 우하단 x (픽셀)
int32 y1    # bbox 좌상단 y (픽셀)
int32 y2    # bbox 우하단 y (픽셀)
```

읽을 때 주의할 점 3가지:

항목	내용
필드 순서	x1, x2, y1, y2 순서입니다. 보통 쓰는 x1,y1,x2,y2 가 아닙니다 . 이름으로 접근하면 문제없지만 순서로 파싱하면 좌표가 섞입니다.
id 의미	<code>yolo_detect.py:246</code> 의 <code>idx_counter</code> — 매 프레임 0부터 다시 셉니다. 프레임 간 동일 객체를 뜻하지 않습니다 . fusion 노드는 이 필드를 아예 안 씁니다.
confidence 없음	YOLO 의 conf 값을 담을 필드가 없습니다. <code>yolo_detect.py:211-221</code> 에서 임계값으로 자른 뒤 값 자체는 버려집니다. → fusion 이 신뢰도 가중치를 쓸 수 없습니다.

Yolo_Objects.msg — 한 프레임 전체

```
std_msgs/Header header # stamp = 원본 CompressedImage 의 stamp (재발행 시각 아님 → 동기화에 필수)
Objects[] yolo_objects # 이 프레임에서 살아남은 bbox 전부
```

`header.stamp` 를 **카메라 원본 stamp 그대로** 복사하는 것이 중요합니다 (`yolo_detect.py:191`). fusion 의 `ApproximateTime` 동기화가 이 stamp 를 기준으로 3개 토픽을 묶기 때문에, 여기서 `rospy.Time.now()` 를 썼다면 YOLO 추론 지연(수십 ms)만큼 어긋나서 동기화가 통째로 깨집니다. **이 부분은 잘 되어 있습니다**.

detected_object.msg / detected_array.msg — 미사용

```
# detected_object.msg
int64 id
geometry_msgs/Pose world_point

# detected_array.msg
std_msgs/Header header
detected_object[] objects
```

전역 좌표(Pose)로 객체를 넘기려던 초기 설계의 흔적입니다. 리포지토리 전체에서 이 두 타입을 `include / import` 하는 코드가 하나도 없습니다. (CMakeLists.txt 의 `add_message_files` 목록에만 등장)

1.4 데이터 흐름

yolo_detect.py		livox_camera_fusion.cpp
Objects()		const auto &Y : yolo->yolo_objects
.id = idx_counter	-X->	(안 씀)
.Class = cls_id	-X->	(안 씀) ← 클래스별 분기 로직 없음
.x1/.y1/.x2/.y2	→	build_scaled_bbox(Y, box)
Yolo_Objects()		
.header (camera stamp)	→	ApproximateTime 동기화 키
.yolo_objects[]	→	루프 대상

yolo_detect.py:46 의 `DEFAULT_PUBLISH_CLASSES = [0]` 때문에 현재 실제로 발행되는 클래스는 **ERP-42(0)** 하나뿐입니다. 그래서 fusion 이 `Class` 를 무시해도 지금은 동작에 문제가 없습니다. 다만 drum/cone 을 켜는 순간 fusion 은 세 클래스를 전부 같은 크기 게이트로 처리하게 됩니다.

1.5 빌드 설정

```
find_package(catkin REQUIRED COMPONENTS
  roscpp          # ← 메시지 전용 패키지에 불필요 (package.xml 에도 선언 안 되어 있음)
  message_generation
  std_msgs geometry_msgs sensor_msgs
)
generate_messages(DEPENDENCIES std_msgs geometry_msgs sensor_msgs)
# ↑ sensor_msgs 는 어떤 msg 도 안 씀
```

- `roscpp` : 메시지 생성에 필요 없습니다. `package.xml` 에는 없는데 CMake 에만 있어 `rosdep` 결과와 실제 빌드 요구사항이 어긋납니다.
- `sensor_msgs` : 4개 msg 중 어느 것도 `sensor_msgs` 타입을 안 씁니다. 불필요한 의존성.
- `geometry_msgs` : `detected_object.msg` 의 `Pose` 때문에만 필요 → 그 msg 를 지우면 같이 제거 가능.

1.6 ⚠ planning_ws 중복 정의 (md5 불일치 지뢰)

planning_ws/src/detect_msgs/ 에 같은 이름, 다른 내용의 패키지가 하나 더 있습니다.

파일	perception_ws	planning_ws
Objects.msg	int32 Class/id/x1/x2/y1/y2	int64 Class/id/x1/x2/y1/y2
Yolo_Objects.msg	std_msgs/Header header	Header header (동일 의미, md5 영향 없음)

- `int32` ↔ `int64` 차이는 메시지 **md5sum** 을 바꿉니다. 한쪽으로 빌드한 publisher 와 다른 쪽으로 빌드한 subscriber 는 ROS 가 `md5sum mismatch` 로 **연결 자체를 거부**합니다.
- 지금 당장은 터지지 않습니다. `detect_msgs` 타입이 워크스페이스 경계를 넘는 토픽이 하나도 없기 때문입니다(planning 은 `sensor_msgs/PointCloud` 만 구독).
- 하지만 `source` 순서에 따라 어떤 `detect_msgs` 가 잡히는지 달라지고, `local_path` 는 [CMakeLists.txt:15](#) 에서 `detect_msgs` 를 의존성으로 걸어놓고도 **소스 어디에서도 쓰지 않습니다.**

정리 방향: `planning_ws/src/detect_msgs` 를 삭제하고 `local_path` 의 `detect_msgs` 의존성도 제거 → 정의를 `perception_ws` 한 곳으로 단일화.

2. livox_clustering

LiDAR 단독 파이프라인. 카메라 없이 포인트클라우드만으로 장애물 중심점을 뽑습니다.

- 노드 이름: `livox_euclidean_clustering`
- 실행: 시스템 `python3 (#!/usr/bin/env python3)` — `numpy / scipy / scikit-learn` 필요
- 파일: `scripts/livox_euclidean_clustering.py` (333줄, 단일 파일)

2.1 코드 구조 지도

```
line 1- 15  import
line 17- 43  모듈 전역 상수 (하드코딩 기본값)
line 45- 84  load_algorithm_params() ← rosparam 으로 위 전역값 덮어쓰기
line 87-105  class KalmanFilter      ← 등속 모델 4상태 KF
line 107-114 class Tracker          ← KF 1개 + miss 카운터 래퍼
line 117-147 parse_xyz_points()     ← PointCloud2 → (N,3) ndarray 고속 파싱
line 149-154 rot_pitch()           ← y축 회전
line 156-159 voxel_downsample()
line 161-168 dror_filter()
line 170-181 grid_ground_remove()
line 183-191 ransac_ground_remove()
line 193-215 dist_euclid_labels()    ← 거리가중 DBSCAN-ish 클러스터링 (핵심)
line 217-229 merge_clusters()
line 231-236 bbox_ok()
line 238-327 class LivoxEuclideanClustering ← ROS 노드
line 330-333 main
```

2.2 콜백 전체 흐름 (`pc_callback` , line 312)

```
def pc_callback(self, msg):
    pts = self._parse_points(msg)          # (N,3) float64
    if pts.size == 0:
        빈 메시지 2개 발행하고 return
    pts = self._preprocess(pts)             # ① ~ ⑥
    self.publish_preprocessed(pts)          # 디버그 토픽
    observed = self._cluster_observations(pts) # ⑦ ~ ⑨ → (M,2)
    self._track(observed)                   # ⑩
    self.publish_centroids([t.last for t in self.trackers.values()])
```

핵심: 최종 발행값은 관측된 클러스터 중심이 아니라 **트래커의 last** 값입니다. 따라서 관측이 없어도 `tracker_max_miss` 프레임 동안은 예측 위치가 계속 발행됩니다(coasting). 반대로 새 물체가 나타난 첫 프레임에도 바로 발행됩니다 (`Tracker.__init__` 에서 `last = c`).

2.3 단계별 알고리즘 상세

① PointCloud2 파싱 — `parse_xyz_points` (line 132)

두 가지 경로가 있습니다.

```
fields = {f.name: f for f in msg.fields}
if x/y/z 중 하나라도 없거나 FLOAT32 가 아니면:
    → 느린 경로: pc2.read_points() 제너레이터 (순수 Python 루프)
else:
    → 빠른 경로: msg.data 를 uint8 로 통째로 frombuffer →
        (N, point_step) 로 reshape → offset 슬라이스 → view('<f4')
```

Livox 드라이버는 `x, y, z` 를 float32 로 내보내므로 **항상 빠른 경로**를 탑니다. `pc2.read_points()` 대비 대략 수십 배 빠릅니다(20k 포인트 기준 수십 ms → 1ms 미만).

`_pointcloud2_rows` (line 117) 는 `row_step != point_step * width` 인 (= 행마다 패딩이 낀) 클라우드도 처리하려는 방어 코드입니다. Livox 는 `height=1` 이라 이 분기는 실제로는 안 탑니다.

마지막에 `np.isfinite` 로 NaN/Inf 제거 → `float64` 승격.

float64 승격은 이후 KDTree 연산 정확도를 위한 것이지만 메모리는 2배가 됩니다. 정확도가 필요 없다면 float32 유지가 더 빠릅니다.

② pitch 보정 — `rot_pitch` (line 149)

$$R(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \quad p' = p \cdot R^T$$
$$\begin{aligned} x' &= \cos\theta \cdot x + \sin\theta \cdot z \\ y' &= y \\ z' &= -\sin\theta \cdot x + \cos\theta \cdot z \end{aligned}$$

y축(횡방향) 기준 회전 = pitch. `pitch_deg = 0.73` 은 yaml 주석에 따르면 **bag 데이터에서 RANSAC 으로 지면 평면을 60프레임 측정한 median (std 0.01°)** 입니다.

목적: Livox 가 앞쪽 아래로 0.73° 기울어 마운트되어 있어서, 보정하지 않으면 - 8 m 앞 지면이

`z = -8 · tan(0.73°) = -0.102 m` 로 기울어 보이고 - 뒤이은 ROI z-cut / grid 지면제거 / bbox 높이 판정이 전부 거리에 따라 흔들립니다.

보정 후 좌표계를 문서에서는 **leveled frame** 이라 부르겠습니다.

⚠ 발행되는 `frame_id` 는 `livox_frame` (= raw frame) 인데 실제 점은 leveled frame 입니다. 0.73° 차이로 RViz 에서는 눈에 안 띄지만 TF 상으로는 부정확합니다.

③ ROI 박스 컷 (line 274)

```
mask = (ROI_X_MIN <= x <= ROI_X_MAX) &
        (ROI_Y_MIN <= y <= ROI_Y_MAX) &
        (ROI_Z_MIN <= z <= ROI_Z_MAX)
```

yaml 기준 `x∈[0, 8]` , `y∈[-2, 2]` , `z∈[-2, 1.5]` → **전방 8 m × 폭 4 m 의 좁은 통로**만 봅니다. 이 단계에서 포인트 수가 보통 90% 이상 줄어들어 이후 파이프라인 속도를 결정합니다.

④ voxel downsample (line 156)

```
idx = np.unique(np.floor(pts / vs), axis=0, return_index=True)[1]
return pts[idx]
```

- 각 0.1 m 격자에서 **첫 번째로 등장한 점 하나**를 남깁니다(무게중심 평균이 아님 → 더 빠름).
- `np.unique(axis=0)` 은 내부적으로 정렬을 하므로 $O(N \log N)$.
- 결과: 밀도가 거리와 무관하게 균일해짐 → 뒤의 DROR/클러스터링이 거리에 덜 민감해집니다.

⑤ DROR — Dynamic Radius Outlier Removal (line 161)

```
rng = ||(x, y)||2 # 수평 거리
radii = clip(MIN_RADIUS + RADIUS_SCALE*rng, MIN, MAX) # 점마다 다른 반경
counts = KDTree(pts).query_ball_point(pts, radii, return_length=True)
keep = (counts - 1) >= MIN_NEIGHBORS # -1 은 자기 자신 제외
```

원래 DROR 의 아이디어는 "멀수록 포인트가 성기니까 판정 반경도 키운다" 인데,

```
radius = 0.1 + 0.1*rng, 단 0.2 로 clip
→ rng ≥ 1.0 m 이면 항상 0.2 로 포화
```

ROI 가 0~8 m 이므로 사실상 거의 모든 점이 고정 반경 0.2 m 를 씹는다. 즉 지금 설정에서 DROR 은 "반경 0.2 m 안에 이웃 3개 이상" 이라는 평범한 ROR(Radius Outlier Removal) 로 퇴화해 있습니다. → 동적 특성을 살리려면 `dror_max_radius` 를 0.5~0.8 로 올려야 합니다.

효과: 빗방울/먼지/링잉 같은 고립 노이즈 제거. 반대로 멀리 있는 작은 물체의 성긴 포인트도 같이 날아갑니다.

⑥-1 grid 기반 지면 제거 (line 170)

```
ix = floor((x - ROI_X_MIN) / GRID_CELL_SIZE)
iy = floor((y - ROI_Y_MIN) / GRID_CELL_SIZE)
key = ix*100000 + iy # 2D 셀 해시
cnt = 셀별 점 개수
min_z = 셀별 최소 z
ground = (cnt >= GRID_MIN_POINTS) & (z - min_z < GRID_MAX_HEIGHT_DIFF)
return pts[~ground]
```

셀 안에 점이 충분히 많고(≥ 10) 그 셀의 최저점에서 0.2 m 이내면 지면으로 간주 하고 버립니다.

⚠ 현재 파라미터 조합에서는 사실상 동작하지 않습니다.

앞 단계에서 voxel 0.1 m 로 다운샘플했기 때문에, 0.2 m $\times$ 0.2 m 셀 안에 들어갈 수 있는 xy 격자 컬럼은 $2 \times 2 = 4$ 개 뿐입니다. 평평한 지면은 컬럼당 z 가 하나이므로 셀당 최대 4점 → `cnt >= 10` 을 절대 만족하지 못합니다.

반대로 `cnt >= 10` 이 성립하는 셀은 한 컬럼에 여러 z 가 쌓인 곳 = 벽/차량 측면 같은 수직 구조물입니다. 이때는 그 물체의 아래쪽 0.2 m 를 깎아버립니다. 의도와 정반대입니다.

고치려면 `grid_cell_size`: 0.5 (→ $5 \times 5 = 25$ 컬럼) 또는 `grid_min_points`: 3~4 로.

`key = ix*100000 + iy` 는 ROI 컷 뒤라서 `ix, iy >= 0` 이 보장되고 `iy < 100000` 이므로 해시 충돌은 없습니다(안전).

⑥-2 RANSAC 잔류 지면 제거 (line 183)

```
ransac = RANSACRegressor(residual_threshold=GROUND_THRESH).fit(X=pts[:,2], y=pts[:,2])
res = |z - ransac.predict(x,y)|
return pts[res > GROUND_THRESH] # 평면에서 0.3 m 넘게 떨어진 점만 남김
```

$z = a \cdot x + b \cdot y + c$ 평면을 최대 합의(consensus)로 찾고, 그 평면 ± 0.3 m 를 통째로 삭제합니다. ⑥-1 이 거의 안 먹기 때문에 실질적인 지면 제거는 전부 이 RANSAC 이 담당합니다.

두 가지 부작용:

1. 높이 0.3 m 이하 물체는 통째로 사라집니다. (라바콘 높이 ~0.45~0.7 m 는 윗부분만 남음)

- 모든 물체의 **아래 0.3 m 가 잘려나갑니다**. 뒤의 `min_height = 0.3` 게이트와 합치면 **실제 물체 높이가 0.6 m 를 넘어야 통과**하게 됩니다.
- ROI 가 좁아 지면 점이 적고 큰 물체가 화면을 채우면, RANSAC 이 **물체 표면을 평면으로 착각**해 물체를 지워버릴 수 있습니다. (`try/except` 로 감싸져 있어 예외는 안 나고 조용히 실패)

⑦ 거리 가중 Euclidean 클러스터링 — `dist_euclid_labels` (line 193)

DBSCAN 과 비슷하지만 **eps 가 점마다 다른** 변형입니다.

```
tree = cKDTree(pts[:, :2])          # ★ 2D (x,y) 트리 - z 무시
for i in range(n):
    if visited[i]: continue
    d0 = BASE_DIST + DIST_SCALE * |x_i|    # 거리 비례 eps
    Q = tree.query_ball_point(pts[i,:2], d0)
    if len(Q) < MIN_CLUSTER_SIZE:         # core point 아님
        visited[i] = True; continue      # ← 영구 폐기
    stack = Q; lbl[Q] = cid; visited[Q] = True
    while stack:                          # region growing (BFS/DFS)
        cur = stack.pop()
        d = BASE_DIST + DIST_SCALE * |x_cur|
        for nb in tree.query_ball_point(pts[cur,:2], d):
            if not visited[nb]:
                visited[nb] = True
                if len(tree.query_ball_point(pts[nb,:2], d)) >= MIN_CLUSTER_SIZE:
                    stack.append(nb)      # nb 도 core → 확장 계속
                lbl[nb] = cid             # core 가 아니어도 라벨은 부여 (border point)
    cid += 1
```

eps 공식: $\text{eps}(x) = 0.1 + 0.05 \cdot |x|$ (yaml 기준)

거리 x	eps
2 m	0.20 m
4 m	0.30 m
6 m	0.40 m
8 m	0.50 m

멀수록 포인트가 성기므로 임계를 키워 한 물체가 여러 조각으로 깨지는 걸 막습니다.

알아둘 특성 3가지:

- 2D 클러스터링입니다.** `pts[:, :2]` 만 트리에 넣기 때문에 z 가 무시됩니다. → 위아래로 겹친 물체(예: 표지판 기둥 + 그 아래 물체)는 하나로 합쳐집니다. 대신 지면 잔여물이 남아있어도 수직으로 이어붙지는 않습니다.
- |x| 만 씁니다.** 반경 $\sqrt{(x^2+y^2)}$ 이 아니라 전방 거리 x 만 봅니다. ROI 가 `y∈[-2,2]` 로 좁아서 차이는 작지만, 옆쪽 물체는 eps 가 과소평가됩니다.
- 표준 DBSCAN 과 다른 점:** core 가 아닌 점을 만나면 `visited[i]=True` 로 표시하고 버립니다(line 203). 이 점은 나중에 다른 클러스터가 확장해 와도 `if not visited[nb]` 에 걸려 **영원히 라벨을 못 받습니다**. → 스캔 순서(포인트 인덱스 순)에 따라 결과가 달라집니다. 즉 **결정론적이지만 순서 의존적**입니다.

성능: 파이썬 루프 + 점당 최소 1회 KD 질의. 다운샘플 후 2~3천 점이면 대략 30~100 ms. `queue_size=1` 이라 밀리면 프레임을 그냥 버립니다(latency 안정에는 유리).

⑧ 클러스터 병합 — `merge_clusters` (line 217)

```
cent = {라벨: 클러스터 무게중심(3D)}
for l1 in uniq:
    if l1 in merged: continue
    rep[l1] = l1
    for l2 in uniq:
        if ||cent[l1][:2] - cent[l2][:2]|| < CLUSTER_MERGE_GAP: # 0.5 m
            rep[l2] = l1; merged.add(l2)
```

중심 간 거리가 0.5 m 미만이면 하나로 합칩니다. 한 물체가 여러 조각으로 쪼개진 걸 되붙이는 용도입니다.

- **1-패스, 비전이적(non-transitive):** A-B 가 가깝고 B-C 가 가까워도 A-C 가 멀면 A,B 만 합쳐지고 C 는 별개로 남을 수 있습니다(반복 순서 의존).
- 중심 기준이라 **길쭉한 물체 두 개가 나란히** 있으면 중심이 멀어 안 합쳐지고, **작은 물체 두 개가 0.5 m 안에** 있으면(라바콘 2개) 하나로 합쳐집니다.

⑨ bbox 크기 게이트 — `bbox_ok` (line 231)

```
xl, yl, zl = np.ptp(x), np.ptp(y), np.ptp(z) # ptp = max - min (AABB 변 길이)
return (MIN_LENGTH < xl < MAX_LENGTH and
        MIN_WIDTH < yl < MAX_WIDTH and
        MIN_HEIGHT < zl < MAX_HEIGHT)
```

yaml 기준: 0.5 < 길이 < 2.5 , 0.5 < 폭 < 2.5 , 0.3 < 높이 < 1.0

- 부등호가 **strict(<)** 입니다. 정확히 0.5 m 는 탈락.
- **높이 상한 1.0 m** 는 꽤 낮습니다. ERP-42 차량(높이 ~1.2 m)은 ⑥-2 RANSAC 이 아래 0.3 m 를 깎아준 덕분에 겨우 통과하는 셈입니다. RANSAC 임계를 낮추면 오히려 차량이 상한에 걸려 탈락할 수 있습니다 — **커플링 주의**.
- 라바콘(0.3×0.3 m)은 `min_length/min_width = 0.5` 에 걸려 **원천적으로 검출 불가**입니다. 현재 설정은 명백히 **차량급 물체 전용**입니다.

통과한 클러스터만 `mean(x, y)` 를 관측값으로 씁니다 (line 283).

⑩ 칼만 추적 — `_track` (line 287)

상태 모델 (`KalmanFilter`, line 87):

```
state x = [px, py, vx, vy]T

A = [1 0 dt 0]      H = [1 0 0 0]
    [0 1 0 dt]      [0 1 0 0]
    [0 0 1 0]      dt = 0.1 (10 Hz 가정, 하드코딩)
    [0 0 0 1]

Q = 5.0 · I4   (프로세스 노이즈 - 매우 큼)
R = 0.1 · I2   (측정 노이즈 - 작음)
P0 = I4
x0 = 0
```

Q=5, R=0.1 의 의미: 예측 직후 $P_{11} \geq 5$ 이므로 칼만 이득 $K_{11} = P_{11}/(P_{11}+0.1) \approx 0.98$. → 거의 측정값을 그대로 따라갑니다. 스무딩 효과가 사실상 없습니다. 이 KF 의 실질적 역할은 (a) ID 유지, (b) 미검출 프레임 coasting 두 가지뿐입니다. 노이즈를 줄이고 싶으면 `q` 를 0.05~0.5 수준으로 낮춰야 합니다.

매칭 (greedy nearest neighbor):

```

preds = [t.predict() for t in trackers.values()]          # 전 트래커 1-step 예측
dist = ||observed[:,None,:] - preds[None,:,:]||          # (M관측 × K트래커)
while True:
    i, j = argmin(dist)
    if dist[i,j] > MATCH_DIST: break                    # 1.5 m
    trackers[keys[j]].update(observed[i])               # 매칭 성사
    dist[i,:] = dist[:,j] = inf                          # 행/열 제거

```

헝가리안이 아니라 **탐욕적(greedy)** 매칭입니다. 전역 최적은 아니지만 $O(MK)$ 로 빠르고 물체가 5~10개 수준이면 차이가 거의 없습니다.

매칭 안 된 트래커 → `t.no_update()` ; `miss > TRACKER_MAX_MISS` 면 삭제
매칭 안 된 관측 → 새 Tracker 생성 (`id = tid_seq++`)

2.4 🐛 발견된 버그: 이중 예측 (double predict)

```

# line 288
preds = np.array([t.predict() for t in self.trackers.values()]) # ← 1회차 예측
...
# line 302-304
for k, t in list(self.trackers.items()):
    if k not in assigned_trk:
        t.no_update()      # → def no_update(self): self.kf.predict(); self.miss += 1
                           #                               ↑ 2회차 예측

```

매칭에 실패한 트래커는 **한 프레임에 상태가 2번 전진**합니다.

- 내부 상태 `x` 가 `2·dt` 만큼 이동 → **coasting 속도가 실질적으로 2배**
- 공분산 `P` 도 `2Q` 만큼 커짐 → 다음 프레임 게인이 더 커짐
- 반면 발행되는 `t.last` 는 `no_update()` 안에서 갱신되지 않아 **1회차 예측값**입니다. → 내부 상태와 발행값이 한 스텝 어긋난 채로 누적됩니다.

동일한 버그가 C++ 쪽에도 있습니다 (`KalmanTracker::miss()` 가 `predict()` 를 또 호출, `livox_camera_fusion.cpp:51-55`). C++ 은 `miss()` 안의 `predict()` 가 `last_pos` 를 갱신하므로 발행값 불일치는 없지만, 2배 전진은 동일합니다.

수정: `no_update()` 에서 `self.kf.predict()` 를 지우고 `self.miss += 1` 만 남기면 됩니다 (예측은 이미 `_track` 도입부에서 전 트래커에 대해 수행했음).

2.5 파라미터 전체 표

설정 파일: `config/livox_clustering.yaml` 로드 위치: `livox_euclidean_clustering.py:45-84` (`launch` 의 `<rosparam command="load">` 로 private ns 에 올라감)

(A) 토픽/프레임

파라미터	기본값	설명
<code>~input_topic</code>	<code>/livox/lidar</code>	입력 PointCloud2
<code>~centroid_topic</code>	<code>/perception/livox/centroids</code>	외부 인터페이스 (planning 구독)
<code>~preprocessed_topic</code>	<code>/perception/livox/preprocessed</code>	디버그용 전처리 결과
<code>~frame_id</code>	<code>livox_frame</code>	출력 frame_id (실제 점은 leveled frame — 2.3② 참고)

(B) 좌표/ROI

파라미터	값	↑ 올리면	↓ 내리면	튜닝 팁
pitch_deg	0.73	지면이 앞쪽으로 더 들러 보임 → 먼 거리 지면 z 상승 → 지면이 ROI z 상한을 넘거나 물체로 오검출	지면이 앞쪽으로 더 내려감 → 먼 물체 밑이 잘리고 grid 지면제거 오동작	bag 에서 RANSAC 으로 재측정하는 게 정석. 마운트 바꿨으면 반드시 다시 재기. $\pm 0.2^{\circ}$ 이상 틀리면 8 m 에서 3 cm 오차
roi_x_max	8.0	더 멀리 봄 → 조기 감지 가능, 포인트 수 증가 → 처리시간 증가 , 원거리 성긴 클러스터 오검출 ↑	근거리만 봄 → 빠르고 안정적이지만 회피 판단이 늦어짐	차속 × 반응시간으로 결정. 20 km/h(5.6 m/s) 기준 8 m 는 약 1.4초 여유
roi_x_min	0.0	차체 바로 앞 점 제외	음수면 뒤쪽 점 포함(무의미)	범퍼 반사가 잡히면 0.3~0.5 로 올릴 것
roi_y_min/max	± 2.0	폭 넓게 봄 → 옆 차선/연석/화단까지 장애물로 인식	좁게 봄 → 인접 차선 차량 놓침, 곡선로에서 앞차 놓침	차폭(1.6 m) + 여유. 곡선 구간이 많으면 ± 3 권장
roi_z_min	-2.0	지면 아래 점 제외	—	사실상 항상 지면 아래라 영향 없음
roi_z_max	1.5	높은 구조물(터널/표지판/나뭇가지) 포함 → 오검출 ↑	낮은 물체만 → 큰 트럭 상단 잘림	센서 높이 기준. 1.5 는 지면 위 약 1.9 m 에 해당(센서고 0.4 m 가정)

(C) 다운샘플 / 노이즈

파라미터	값	↑ 올리면	↓ 내리면	튜닝 팁
voxel_size	0.1	점 수 급감 → 빨라짐 . 대신 작은 물체 형상 소실, 클러스터 최소 크기 조건 못 맞춤	점 유지 → 정밀하지만 느려짐 (전체 처리시간을 지배), grid 지면제거 셀 통계는 오히려 개선	0.05~0.15 범위. 이 값은 grid_cell_size , euclidean_min_cluster_size 와 강하게 커플링
dror_min_neighbors	3	노이즈 강하게 제거, 원거리 실물체도 같이 삭제	노이즈 통과 → 허위 클러스터 발생	voxel 0.1 / radius 0.2 조합에서 표면 위 이웃은 대략 4~8개. 3 은 적당, 5 이상은 위험
dror_min_radius	0.1	근거리 판정 반경 ↑ → 관대	반경 ↓ → 근거리 점도 노이즈로 삭제	voxel_size 이상이어야 의미가 있음
dror_radius_scale	0.1	거리 비례 증가율 ↑ → 원거리 관대	원거리 엄격	max_radius 에 걸려 있어 현재는 효과 거의 없음
dror_max_radius	0.2	DROR 이 실제로 "동적"이 됨. 원거리 물체 보존 ↑, 노이즈 통과 ↑	전 구간 고정 반경	현재 $\text{rng} \geq 1\text{ m}$ 에서 포화. 0.5~0.8 로 올려야 원래 의도대로 동작

(D) 지면 제거

파라미터	값	↑ 올리면	↓ 내리면	튜닝 팁
grid_cell_size	0.2	셀당 점 증가 → grid_min_points 를 만족하기 시작 → 지면 제거가 실제로 동작 . 대신 경사면/연석을 지면으로 오판	셀당 점 감소 → 더더욱 동작 안 함	0.4~0.5 권장 (voxel 0.1 기준 16~25 컬럼)
grid_max_height_diff	0.2	최저점 위 더 두꺼운 층을 지면으로 삭제 → 물체 밑동 손실 ↑	얇은 층만 삭제 → 울퉁불퉁한 지면 잔류	노면 거칠기 + 센서 노이즈 수준(2~3 cm)의 몇 배
grid_min_points	10	지면 판정이 더 까다로워짐 → 아무것도 안 지워짐	3~4 로 낮추면 지면 제거가 살아남	현재 조합(0.2 셀 / 0.1 voxel)에서 이론 최대 4점 → 10은 불가능한 값
ground_thresh	0.3	평면 ± 0.3 초과만 남김 → 더 두껍게 삭제 : 낮은 물체 소멸, 물체 밑동 손실 ↑, 대신 지면 잔류 없음	얇게 삭제 → 지면 잔류 → 지면이 물체들을 연결해 거대 클러스터 생성	min_height 와 합이 실제 필요 물체 높이. 현재 $0.3+0.3=0.6\text{ m}$

(E) 클러스터링

파라미터	값	↑ 올리면	↓ 내리면	튜닝 팁
euclidean_base_dist	0.1	근거리 eps ↑ → 인접 물체 병합(과분할 감소, 오병합 증가)	근거리 eps ↓ → 한 물체가 여러 조각으로 분할	voxel_size 의 1~2배가 하한. 0.1 은 voxel 0.1 기준 최소값에 가까움
euclidean_dist_scale	0.05	원거리 eps 급증 → 8 m 에서 0.05→0.1 로 바꾸면 eps 0.5→0.9 m, 옆 차선 물체까지 병합	원거리 분할 심화	$eps(x)=base+scale \cdot x$. 8 m 에서 0.5 m 가 현재 값
euclidean_min_cluster_size	5	core point 조건 강화 → 성긴 원거리 물체 탈락, 노이즈 강건	노이즈 몇 점만으로 클러스터 생성	voxel 후 점 수 기준. 8 m 라바콘은 voxel 후 5~10점 수준
cluster_merge_gap	0.5	넓게 병합 → 나란한 두 물체가 하나로(중심이 두 물체 사이 허공에 찍힘)	병합 안 함 → 한 물체가 중심 여러 개로 발행 → planning 이 과잉 회피	물체 최소 간격의 절반 이하로

(F) 크기 게이트

파라미터	값	↑ 올리면	↓ 내리면
min_length / min_width	0.5 / 0.5	작은 물체 제외 (노이즈 강건 ↑, 라바콘·사람 놓침)	작은 물체 허용 (라바콘 검출 가능, 노이즈 클러스터도 통과)
max_length / max_width	2.5 / 2.5	큰 물체 허용 (벽/가드레일 통째로 장애물화 → 경로 막힘)	큰 물체 제외 (근거리 차량이 크게 보여 탈락할 수 있음)
min_height	0.3	높이 낮은 잔류 지면 제거 ↑, 낮은 물체 놓침	낮은 물체 검출, 지면 잔류물이 물체로
max_height	1.0	큰 차량/트럭 허용	벽·기둥 제외. 1.0 은 이미 낮아서 ERP-42 도 아슬아슬

 라바콘/드럼을 잡고 싶다면 최소한 min_length: 0.15, min_width: 0.15, min_height: 0.15 + ground_thresh: 0.15 로 낮춰야 합니다. 대신 노이즈 클러스터가 늘어나므로 euclidean_min_cluster_size 를 함께 올려 균형을 잡으세요.

(G) 트래커

파라미터	값	↑ 올리면	↓ 내리면	튜닝 팁
match_dist	1.5	큰 점프도 같은 물체로 인정 → ID 유지 ↑, 다른 물체끼리 오매칭	조금만 튀어도 새 ID → ID 스위칭 빈발, 유령 트래커 증식	자차 속도 × dt + 물체 속도 × dt. 20 km/h @10 Hz = 0.56 m/frame → 1.5 는 여유 있음
tracker_max_miss	5	오래 coasting → 사라진 물체가 최대 0.5초 더 발행됨 (유령 장애물)	즉시 삭제 → 한 프레임 미검출에도 깜빡임(planning 이 회피/복귀 반복)	5 프레임 = 0.5 s. 3~5 가 적당
KF q (하드코딩 5.0)	5.0	측정을 더 신뢰 → 반응 빠름, 노이즈 그대로 통과	예측을 더 신뢰 → 부드럽지만 급기동에 지연	현재 5.0 은 사실상 스무딩 무효화. 0.05~0.5 권장
KF r (하드코딩 0.1)	0.1	측정을 덜 신뢰 → 부드러움	측정 신뢰 → 노이즈 통과	LiDAR 중심점 지터(수 cm) 수준으로
KF dt (하드코딩 0.1)	0.1	—	—	LiDAR 실제 주기와 맞아야 함. Livox 가 10 Hz 가 아니면 속도 추정이 통째로 틀림

2.6 죽은 코드 / 정리 대상

위치	내용	판정
<code>Tracker.id</code> (line 109)	<code>self.id = tid</code> 로 저장하지만 어디서도 읽지 않음 . 출력은 익명 점 리스트	죽은 필드
<code>KalmanFilter.__init__(q, r)</code> (line 88)	<code>Tracker</code> 가 <code>dt</code> 만 넘기므로 <code>q, r</code> 은 항상 기본값 . 파라미터화된 척만 함	사실상 하드코딩
<code>LivoxEuclideanClustering._parse_points</code> (line 269)	<code>parse_xyz_points(msg)</code> 를 그대로 부르는 1줄 래퍼	불필요한 간접층
<code>parse_xyz_points</code> 느린 경로 (line 136-137)	Livox 는 항상 float32 x/y/z → 실행되지 않음	방어 코드 (유지 무해)
<code>_pointcloud2_rows</code> 의 다중 row 처리 (line 125-130)	Livox 는 <code>height=1</code> → 실행되지 않음	방어 코드 (유지 무해)
모듈 전역 상수 17-43	<code>load_algorithm_params()</code> 가 전부 덮어씀. 단 yaml 없이 rosrun 하면 이 값이 그대로 쓰이고, yaml 과 값이 다름 ⚠	아래 표 참고
<code>import time</code> / <code>rospy.logdebug</code> (line 313, 327)	<code>logdebug</code> 라 기본 로그 레벨에서 안 보임	유지 무해

⚠ 코드 기본값 vs yaml 값 불일치 — `rosrun` 으로 띄우면 완전히 다른 동작을 합니다:

파라미터	코드 기본값	yaml 값	차이
<code>PITCH_DEG</code>	0.1	0.73	지면 보정이 거의 안 됨
<code>ROI_X_MAX</code>	12	8	1.5배 먼 거리까지 봄
<code>ROI_Y_MIN/MAX</code>	±4	±2	폭 2배
<code>EUCLIDEAN_BASE_DIST</code>	0.05	0.1	클러스터가 절반으로 잘게 쪼개짐

전역 상수를 yaml 값과 동기화하거나, 아예 `rospy.get_param` 에 기본값을 안 주고 없으면 죽게 만드는 편이 안전합니다.

3. livox_camera_fusion

LiDAR + 카메라 융합 파이프라인. YOLO 가 "어디에 무엇이 있는지"를 픽셀로 알려주면, 그 픽셀 영역에 투영되는 LiDAR 점만 골라 3D 위치를 계산합니다.

- 노드 이름: `livox_camera_fusion` / 실행파일 `livox_camera_fusion_node`
- 파일: `src/livox_camera_fusion.cpp` (485줄), `include/livox_camera_fusion.h` (151줄), `src/main.cpp` (11줄)

3.1 초기화 (`LivoxCameraFusion::LivoxCameraFusion` , line 62)

```
ros::NodeHandle nh("~"); // main.cpp - private namespace
LivoxCameraFusion node(&nh);
```

모든 파라미터가 **private ns(~)** 로 읽힙니다. launch 의 `<param>` / `<rosparam>` 이 `<node>` 태그 **안에** 있으므로 일치합니다.

```
sub_lidar = Subscriber<PointCloud2>(nh, lidar_topic, 10);
sub_camera = Subscriber<CompressedImage>(nh, camera_topic, 10);
sub_yolo = Subscriber<Yolo_Objects>(nh, yolo_topic, 10);

sync = Synchronizer<ApproximateTime<PointCloud2, CompressedImage, Yolo_Objects>>(
    SyncPolicy(20), *sub_lidar, *sub_camera, *sub_yolo);
sync->setMaxIntervalDuration(ros::Duration(0.05)); // slop 50 ms
```

3개 토픽 동기화가 이 노드의 전제 조건입니다.

- `ApproximateTime` 은 세 토픽의 stamp 가 **50 ms 이내**로 모이는 조합만 콜백에 넘깁니다.
- 큐 20개 × 3토픽. 어느 하나라도 끊기면 **콜백이 아예 안 불립니다**.
- YOLO 추론이 느려서 카메라(30 Hz)와 YOLO(예: 8 Hz)의 stamp 간격이 벌어지면 동기화 성공률이 급락합니다. **디버깅 1순위 지점**.

```
read_projection_matrix(); // ~camera_matrix/data (3x3), ~extrinsic_matrix/data (3x4)
projection_matrix = K * T; // 3x4
```

`config/projection.yaml` 이 없거나 크기가 안 맞으면 `ROS_WARN` 후 **코드 내장 기본값**을 씁니다(line 473-482). 내장값은 `yaml` 과 **수치가 동일**합니다 → 설정이 두 곳에 중복 존재. 한쪽만 고치면 조용히 어긋나므로 관리 위험 요소입니다.

3.2 투영 수학 (핵심)

K (intrinsic, 3x3)	T (extrinsic, 3x4)
$\begin{bmatrix} 1974.0 & 0 & 963.6 \\ 0 & 1971.8 & 584.9 \\ 0 & 0 & 1.0 \end{bmatrix}$	$\begin{bmatrix} 0.0026 & -1.0000 & -0.0036 & 0.0070 \\ 0.0108 & 0.0036 & -0.9999 & -0.0679 \\ 0.9999 & 0.0026 & 0.0108 & 0.1865 \end{bmatrix}$

$P = K \cdot T \quad (3 \times 4)$

$$\begin{bmatrix} u' \\ v' \\ w' \end{bmatrix} = P \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \rightarrow (u, v) = (u'/w', v'/w')$$

extrinsic 이 뭘 하는지 읽어보면:

```

x_cam ≈ -y_lidar + 0.0070    (카메라 오른쪽 = LiDAR 왼쪽)
y_cam ≈ -z_lidar - 0.0679    (카메라 아래 = LiDAR 위)
z_cam ≈ x_lidar + 0.1865     (카메라 광축 = LiDAR 전방)

```

전형적인 LiDAR→카메라 축 변환입니다. 평행이동에서 - +0.1865 : 카메라 원점이 LiDAR 보다 **0.1865 m 뒤** - -0.0679 : 카메라가 LiDAR 보다 **0.0679 m 위** (y_cam 은 아래가 +)

cv::perspectiveTransform(3채널 입력, 2채널 출력, 3x4 행렬) 은 OpenCV 가 공식 지원합니다
(src.channels() + 1 == m.cols , dst.channels() == m.rows - 1).

보정 순서가 중요합니다 (line 114-136 주석에도 명시):

- ① raw frame 그대로 투영 ← extrinsic 이 마운트 기울기를 이미 흡수하고 있음
- ② 그 다음 leveled frame 으로 회전 → ROI 컷

즉 투영에는 **pitch 보정을 적용하지 않고**, 3D ROI/크기 판정에만 적용합니다. kept_points / kept_proj 두 배열을 같은 인덱스로 동기 유지하며 필터링하는 부분 (line 124-136)이 이 설계의 핵심입니다. 잘 짜여 있습니다.

3.3 콜백 흐름 (detectionCallback , line 95)

```

camera_image = cv_bridge::toCvCopy(cam_msg, "bgr8")->image;    // JPEG 디코딩
pcl::fromROSMsg(*lidar_msg, *pc);                               // PointXYZI (intensity 는 버림)

lidar_points ← pc 의 (x,y,z) 전부                                // 다운샘플/노이즈 제거 없음!
cv::perspectiveTransform(lidar_points, projected_list, projection_matrix);

for each point:                                                  // leveled 회전 + ROI
    x_lvl = cosθ·x + sinθ·z
    z_lvl = -sinθ·x + cosθ·z
    ROI: x_lvl∈[0,15], y∈[-7,7], z_lvl∈[-2,2]
    통과하면 kept_points / kept_proj 에 push (인덱스 동기)

convert_msg(yolo_msg, lidar_msg->header);

```

livox_clustering 과 결정적으로 다른 점: voxel downsample, DROR 이 없습니다. "YOLO bbox 가 이미 강력한 필터니까 전처리 불필요" 라는 설계입니다. 대신 원본 2만 점 전부에 대해 투영 + ROI 를 돌립니다(그래도 O(N) 벡터 연산이라 빠름).

3.4 객체별 처리 (convert_msg , line 144)

각 YOLO bbox 마다 6단계 게이트를 통과해야 centroid 가 나옵니다.

```

for (Y : yolo->yolo_objects)
    ① build_scaled_bbox(Y, box)                                실패 → skip
    ② collect_points_in_bbox(box, matched_px, local)
        matched_px.size() < CLUSTER_MIN_SIZE(3)              → skip
    ③ extract_roi(matched_px, local, box.center)
        roi->size() < CLUSTER_MIN_SIZE(3)                     → skip
    ④ remove_ground_from_cloud(roi)
        roi_ng->empty()                                         → skip
    ⑤ largest_cluster_centroid(roi_ng, centroid, ext)
        클러스터 없음                                          → skip
    ⑥ 3D AABB 게이트 (length/width/height)                    → skip
        통과 → cur_centroids.push_back(centroid)
            out_cloud += roi_ng

```


① build_scaled_bbox (line 200)

```
cx = (x1+x2)/2 ; cy = (y1+y2)/2
hw = (x2-x1)/2 * BBOX_SCALE_RATIO // 1.0 → 스케일링 없음
hh = (y2-y1)/2 * BBOX_SCALE_RATIO
box.x1 = max(0, cx-hw) // 이미지 경계로 clamp
box.x2 = min(cols-1, cx+hw)
return (box.x2-box.x1) >= MIN_BBOX_EDGE_PX(0) && (box.y2-box.y1) >= 0
```

MIN_BBOX_EDGE_PX = 0 이라 "폭/높이 ≥ 0 " 조건인데, **완전히 무의미하지는 않습니다**: bbox 가 이미지 밖으로 완전히 벗어나면 clamp 후 $x_2 - x_1 < 0$ 이 되어 걸러집니다. 다만 1픽셀짜리 bbox 도 통과합니다.

camera_image.cols/rows 를 쓰기 위해 **매 프레임 JPEG 을 디코딩**합니다. 1920×1200 JPEG 디코딩은 5~15 ms — 이미지 크기 상수 2개를 위해 지불하는 비용치고 큼니다.

② collect_points_in_bbox (line 218)

```
for (i : projected_list)
    if (isnan(u) || isnan(v)) continue; // ← 실질적으로 발생 안 함 (아래 참고)
    if (box.x1 <= u <= box.x2 && box.y1 <= v <= box.y2)
        matched_px.push_back({u,v});
        local->push_back(lidar_points[i]); // 인덱스 동기 활용
```

- **후방 점 문제 없음**: extrinsic 3행이 $w' \approx x_{\text{lidar}} + 0.1865$ 이고 ROI 에서 $x_{\text{lvl}} \geq 0$ 을 이미 걸렀으므로 $w' > 0$ 이 보장됩니다.
- **isnan** 체크: OpenCV 는 $w'==0$ 일 때 좌표를 0 으로 만들지 NaN 을 만들지 않습니다. → 이 분기는 거의 dead. 무해하지만 실제 방어는 못 합니다.
- **오클루전 처리 없음**: bbox 안에 앞차 뒤의 벽/지면이 같이 잡힙니다. 이걸 뒤의 ③/⑤ 가 걸러야 합니다.

③ extract_roi (line 238) — △ 이 노드의 최대 병목

```
if (cv::norm(matched_px[i] - center) <= ROI_RADIUS_PX) // 10.0 px
    roi->push_back(local->points[i]);
```

bbox 중심에서 반경 10 픽셀 원 안에 투영되는 점만 남깁니다.

이게 얼마나 좁은지 계산해 보면:

```
실제 폭 [m] = 픽셀 폭 [px] × 거리 d [m] / fx
              = 20 px × d / 1974
              = 0.01013 · d
```

거리 d	원 지름이 덮는 실제 폭
3 m	0.030 m (3 cm)
5 m	0.051 m
10 m	0.101 m
15 m	0.152 m

즉 5 m 앞 차량에서 **가로세로 5 cm** 짜리 패치만 보는 셈입니다. BBOX_SCALE_RATIO 를 아무리 바꿔도 중심은 안 움직이므로 **효과가 없습니다**(dead parameter). 사실상 bbox 는 "중심 좌표"로만 쓰이고 있습니다.

④ remove_ground_from_cloud → remove_ground_ransac (line 388) — ⚠ 무동작

```
const auto keep = remove_ground_ransac(tmp, GROUND_THRESH); // GROUND_THRESH = 0.0
```

GROUND_THRESH = 0.0 을 코드에 대입해 따라가 보면:

```
if (pts.size() < 10) return 전체 인덱스; // 10점 미만이면 그냥 다 통과

for (iter = 0; iter < 30; ++iter) {
    ... 세 점으로 평면 (a,b,c) 후보 생성 ...
    for (p : pts)
        if (|p.z - (a*p.x + b*p.y + c)| < 0.0) ++in; // ← 절대값 < 0 : 절대 참이 될 수 없음
        if (in > max_in) { ... } // ← 0 > 0 : 절대 참이 될 수 없음
}
// 따라서 a = b = c = 0 (초기값 그대로)

for (i : pts)
    if (|pts[i].z - 0| > 0.0) idx.push_back(i); // z != 0.0 인 점 전부 통과
```

결론: 지면 제거가 완전히 무효화되어 있습니다. $z == 0.0$ 인 점(존재하지 않음)만 제거합니다. 30회 RANSAC 반복은 순수한 낭비 연산입니다.

부수적으로: - `std::default_random_engine gen;` 이 매 호출마다 기본 시드로 새로 생성됩니다. → 모든 객체·모든 프레임이 정확히 같은 난수열을 씁니다. 재현성은 좋지만 RANSAC 의 의미가 퇴색. - `pts.size() < 10` 일 때 지면 제거를 통째로 건너웁니다. ③에서 점이 많이 줄어드니 이 분기를 타는 경우가 흔합니다.

⑤ largest_cluster_centroid (line 269)

```
pcl::EuclideanClusterExtraction<pcl::PointXYZ> ec;
ec.setClusterTolerance(CLUSTER_TOLERANCE); // 0.4 m - 3D 거리
ec.setMinClusterSize(CLUSTER_MIN_SIZE); // 3
ec.setMaxClusterSize(CLUSTER_MAX_SIZE); // 100 ← 상한 주의
ec.extract(clusters);
→ 점 개수가 가장 많은 클러스터 선택
→ centroid = (mean x, mean y) // z 는 버림
→ extent = (xmax-xmin, ymax-ymin, zmax-zmin)
```

- 여기서 **오류루전이 걸러집니다**: 좁은 원뿔 안에 앞 물체(5 m)와 배경(12 m)이 같이 있으면 0.4 m 톨러런스로는 연결되지 않으므로 별개 클러스터가 되고, 점이 많은 쪽이 선택됩니다.
- `setMaxClusterSize(100)` : 100점을 넘는 클러스터는 결과에서 제외됩니다. ③ 때문에 100점을 넘길 일이 거의 없지만, ROI 반경을 키우면 즉시 문제가 됩니다.
- 중심은 **무게중심(mean)** 이라 표면 점 분포에 치우칩니다(물체 중심이 아니라 "보이는 면의 중심").

⑥ 3D AABB 게이트 (line 176-180) — ⚠ ③과 충돌

```
length = ext[0] // x (깊이 방향)
width = ext[1] // y (횡방향)
height = ext[2] // z (수직)
if (length < 0.1 || length > 10.0) continue;
if (width < 0.1 || width > 10.0) continue;
if (height < 0.1 || height > 10.0) continue;
```

헤더 주석은 "현재는 MIN만 0.05로 노이즈 컷, MAX는 사실상 disabled(10m)"라고 되어 있지만 실제 코드값은 0.05 가 아니라 0.1 입니다(주석-코드 불일치).

그리고 ③의 계산과 합치면 심각한 상호작용이 나옵니다:

```
width ≤ 20 px × d / fx = 0.01013 · d    (원 안에 갇혀 있으므로 물리적 상한)
height ≤ 20 px × d / fy = 0.01014 · d

width ≥ 0.1 을 만족하려면 → d ≥ 9.87 m
height ≥ 0.1 을 만족하려면 → d ≥ 9.86 m
```

LiDAR ROI 가 $x \in [0, 15]$ 이므로, 이 노드는 구조적으로 약 10~15 m 구간의 물체만 출력할 수 있습니다. 10 m 이내 물체는 width/height 게이트에서 전부 탈락합니다.

이건 계산상의 결론이므로 실제 bag 으로 반드시 확인해 보세요. 확인 방법은 7. 디버깅 치트시트 참고. 만약 실제로 근거리 검출이 안 되고 있었다면 원인은 십중팔구 여기입니다.

3.5 추적 (track_and_visualize , line 370) — ⚠ 출력에 영향 없음

```
void LivoxCameraFusion::convert_msg(...) {
    ...
    prev_centroids = cur_centroids;
    track_and_visualize(prev_centroids);    // ① 트래커 갱신 + 이미지에 그림
    publish_2D_pointcloud(prev_centroids, header);    // ② ← 원본 관측값을 그대로 발행
}
```

track_and_visualize 의 결과가 publish_2D_pointcloud 에 전혀 반영되지 않습니다. 발행되는 건 그 프레임의 생 관측값 (cur_centroids) 입니다.

따라서 fusion 노드에서 칼만 트래커가 하는 일은: - X 노이즈 스무딩 — 없음 - X 미검출 프레임 coasting — 없음 (검출이 없으면 그냥 빈 메시지 발행) - X ID 유지 — 출력에 ID 필드가 없음 - ○ camera_image 에 원 + 숫자 그리기 — 그런데 그 이미지를 아무도 안 봄

camera_image 는 cv::imshow 도 안 하고 publish 도 안 합니다. → KalmanTracker 클래스 전체, match_and_update_trackers , draw_bbox_debug , track_and_visualize 가 전부 죽은 코드입니다.

livox_clustering 은 t.last (트래커 상태)를 발행하는데 fusion 은 생 관측값을 발행합니다. 두 파이프라인의 출력 성격이 다르다는 점을 planning 쪽에서 알고 있어야 합니다.

3.6 발행 (publish_2D_pointcloud , line 435)

```
sensor_msgs::PointCloud cloud;
cloud.header = hdr;    // ← LiDAR 원본 stamp (중음)
cloud.header.frame_id = frame_name;    // "livox_frame"
for (p : pts) cloud.points.push_back({p.x, p.y, 0});
ChannelFloat32 ch; ch.name = "dummy"; ch.values = 1.0f × N;    // 사용처 없음
centroid_pub.publish(cloud);
```

- stamp 가 LiDAR 원본입니다. livox_clustering 이 rospy.Time.now() 를 쓰는 것과 대조적으로 이쪽이 올바릅니다.
- "dummy" 채널은 아무도 안 읽습니다(obstacle_manager.cpp:145 는 msg->points 만 순회).
- 검출이 0개여도 빈 PointCloud 를 매 프레임 발행합니다 → planning 의 is_ready 가 계속 true 로 유지되는 효과.

filtered_cloud_pub 은 out_cloud 가 비어있지 않을 때만 발행합니다(line 191). 즉 검출이 없으면 디버그 토픽이 아예 안 나와서 RViz 에 이전 프레임이 남습니다. "멈춘 것처럼 보이는" 착시의 원인이 될 수 있습니다.

3.7 파라미터 전체 표

fusion 은 알고리즘 파라미터가 전부 헤더의 `constexpr` 입니다. 바꾸려면 **재컴파일**이 필요합니다 (rosparam/dynamic_reconfigure 로 못 바꿈).

(A) ROS 파라미터 (launch 로 변경 가능)

파라미터	기본값	설명
<code>~lidar_topic</code>	<code>/livox/lidar</code>	입력 LiDAR
<code>~camera_topic</code>	<code>/camera/image_raw/compressed</code>	입력 카메라
<code>~yolo_topic</code>	<code>/perception/camera/yolo</code>	YOLO bbox
<code>~centroid_topic</code>	<code>/perception/fusion/centroids</code>	외부 인터페이스
<code>~filtered_cloud_topic</code>	<code>/perception/fusion/filtered_cloud</code>	디버그
<code>~frame_name</code>	<code>livox_frame</code>	출력 frame_id
<code>~camera_matrix/data</code>	projection.yaml	K (3×3, row-major 9개)
<code>~extrinsic_matrix/data</code>	projection.yaml	T (3×4, row-major 12개)

(B) 동기화 (cpp 하드코딩)

위치	값	↑ 올리면	↓ 내리면
<code>SyncPolicy(20)</code>	20	큐 깊어짐 → 매칭 성공률 ↑, 지연 ↑, 메모리 ↑	지연 ↓, stamp 가 조금만 엇갈려도 프레임 통째로 버림
<code>setMaxIntervalDuration</code>	0.05 s	slop 완화 → 콜백 자주 발생, LiDAR·카메라 시간차만큼 투영 오차 (20 km/h 에서 50 ms = 0.28 m)	엄격 → 정확하지만 콜백이 거의 안 불릴 수 있음
Subscriber 큐	10	유실 방지	최신성 우선

노드가 조용하면 **가장 먼저 의심할** 곳입니다. `rostopic hz` 로 세 토픽의 주기와 `rostopic echo --noarr .../header` 로 stamp 를 비교하세요.

(C) 알고리즘 상수 (헤더 — 재컴파일 필요)

상수	값	↑ 올리면	↓ 내리면	비고
<code>BBOX_SCALE_RATIO</code>	1.0	bbox 확대 → 주변 점 더 포함	bbox 축소	ROI_RADIUS_PX 가 지배하므로 사실상 효과 없음
<code>MIN_BBOX_EDGE_PX</code>	0	작은 bbox 제거 → 원거리 객체 탈락	—	화면 밖 bbox 제거 기능만 남음
<code>ROI_RADIUS_PX</code>	10.0	점 많이 확보 → 근거리 검출 가능해짐 , 배경/지면 혼입 ↑	더 좁아짐 → 점 부족으로 전부 탈락	★ 가장 영향 큰 파라미터 . bbox 크기 비례(예: $0.25 \times \text{bbox_h}$)로 바꾸는 게 정석
<code>GROUND_THRESH</code>	0.0	평면 $\pm th$ 삭제 → 지면/노면 제거 동작 시작, 낮은 물체 소실	—	★ 0 이라 지면 제거 무효 . 0.1~0.2 권장
<code>CLUSTER_TOLERANCE</code>	0.4	멀리 떨어진 점도 연결 → 앞 물체와 배경이 한 덩어리로 (오클루전 방어 실패)	잘게 분할 → 최대 클러스터가 물체 일부만	3D 거리 기준

상수	값	↑ 올리면	↓ 내리면	비고
CLUSTER_MIN_SIZE	3	노이즈 강건, 원거리·소형 객체 탈락	점 2개로도 클러스터 → 오검출	②③에서 게이트로도 쓰임
CLUSTER_MAX_SIZE	100	큰 클러스터 허용	큰 클러스터 통째로 제외	ROI 변경 키우면 반드시 같이 올려야 함
CLUSTER_MIN_LENGTH/WIDTH/HEIGHT	0.1	노이즈 컷 강화	작은 클러스터 허용	★ ROI_RADIUS_PX 와 충돌 (3.4⑥ 참고). 0.02 정도로 낮추거나 ROI 반경을 키울 것
CLUSTER_MAX_LENGTH/WIDTH/HEIGHT	10.0	—	큰 물체 제외	사실상 비활성
LIDAR_PITCH_DEG	0.73	지면이 앞쪽으로 들림	지면이 앞쪽으로 내려감	livox_clustering.yaml 과 같은 값 — 마운트 바꾸면 두 곳 다 고쳐야 함
LIDAR_ROI_X_MIN/MAX	0 / 15	더 멀리	근거리만	livox_clustering(8 m)보다 넓음
LIDAR_ROI_Y_MIN/MAX	±7	넓게	좁게	livox_clustering(±2)보다 훨씬 넓음. YOLO 가 필터하니 관대해도 OK
LIDAR_ROI_Z_MIN/MAX	-2 / 2	높은 구조물 포함	낮은 것만	
MATCH_DIST	0.7	ID 유지 ↑, 오매칭 ↑	ID 스위칭 ↑	주석대로 20 km/h@10 Hz 기준 잘 잡힘. 다만 출력에 영향 없음
TRACKER_MAX_MISS	10	오래 coasting	빨리 삭제	출력에 영향 없음
KF processNoiseCov	5e-2	측정 신뢰	예측 신뢰	출력에 영향 없음
KF measurementNoiseCov	1e-1	부드러움	반응 빠름	출력에 영향 없음
KF dt	0.1 f	—	—	10 Hz 가정

3.8 죽은 코드 / 정리 대상

위치	내용
cfg/LidarClustering.cfg	완전히 죽은 파일. dynamic_reconfigure::Server 를 만드는 코드가 소스에 없고, 생성된 LidarClusteringConfig.h 를 include 하는 곳도 없음. 게다가 PACKAGE = "Object_detect" 로 다른 프로젝트 이름이 박혀 있고, 파라미터 기본값도 실제 코드와 전혀 다름(PITCH_DEG 3.3 vs 실제 0.73). 다른 프로젝트에서 복붙된 잔재. → 파일 + CMakeLists.txt:23-25 + package.xml 의 dynamic_reconfigure 의존성까지 삭제 가능
KalmanTracker 클래스 전체	출력에 영향 없음 (3.5 참고)
match_and_update_trackers / track_and_visualize	위와 동일
draw_bbox_debug (line 313)	camera_image 에 사각형/원을 그리지만 그 이미지는 어디에도 안 나감
KalmanTracker::id 필드	id 를 저장하지만 화면 출력은 std::to_string(kv.first) (map 키)를 사용 → 필드는 읽히지 않음
remove_ground_ransac 의 30회 루프	th=0.0 때문에 결과에 영향 0. 순수 낭비 연산
remove_ground_ransac 기본인자 threshold = GROUND_THRESH	항상 명시적으로 넘겨서 기본값 미사용
match_and_update_trackers 기본인자	동일

위치	내용
<code>collect_points_in_bbox</code> 의 <code>isnan</code> 체크	OpenCV 가 NaN 을 만들지 않아 실질 dead
<code>prev_centroids</code> 멤버	대입 직후 두 줄에서만 쓰이는 사실상 지역변수
"dummy" 채널 (line 449-452)	소비자 없음
<code>read_projection_matrix</code> 내장 기본 행렬 (line 473-482)	<code>projection.yaml</code> 과 값이 완전히 동일 → 설정 이중화
cpp 상단 중복 include	<code>pcl/segmentation/extract_clusters.h</code> , <code>pcl/search/kdtree.h</code> , <code>sensor_msgs/*</code> 가 헤더에도 cpp 에도 있음
<code>CMakeLists.txt</code> 의 <code>rospy</code>	C++ 전용 패키지인데 의존성에 들어있음
<code>pcl::PointXYZI</code> 의 <code>intensity</code>	읽고 나서 버림. <code>PointXYZ</code> 로 충분

3.9 🐛 미묘한 문제들

1. `trackers[next_tracker_id] = KalmanTracker(cents[i], next_tracker_id++)` (line 367)
`trackers[next_tracker_id]` (좌변)와 `next_tracker_id++` (우변)의 평가 순서가 C++14 에서 **미정(unspecified)** 입니다. 컴파일러에 따라 map 키가 N 또는 N+1 이 되어 `KalmanTracker::id` 와 어긋납니다. 지금은 id 가 화면 출력에만 쓰여서 영향이 없지만 나중에 ID 를 발행하기 시작하면 바로 터집니다.
2. PCL 클라우드의 **width / height 미갱신** `extract_roi` / `remove_ground_from_cloud` 는 `cloud->points.push_back()` 만 하고 `width / height` 를 세팅하지 않습니다. 지금은 `operator+=` 가 `out_cloud` 의 값을 바로잡아 주기 때문에 `toROSMsg` 결과가 정상입니다. 하지만 만약 `roi_ng` 를 직접 `toROSMsg` 로 발행하면 **점 0개짜리 메시지**가 나옵니다. 리팩터링 시 지뢰.
3. 좌표계 표기 불일치 `filtered_cloud` 는 leveled frame 점인데 header 는 `livox_frame (raw)`. 0.73° 차이.
4. `out_cloud` 가 비면 발행 안 함 → RViz 에 이전 프레임 잔상 (3.6 참고).
5. 전역 `static std::map<int, KalmanTracker> trackers;` (line 58) 클래스 멤버가 아니라 파일 스코프 전역. 인스턴스가 하나뿐이라 지금은 문제없지만 nodelet 화하거나 인스턴스를 2개 만들면 트래커가 뒤섞입니다.

4. 두 파이프라인 비교

항목	livox_clustering	livox_camera_fusion
언어	Python3	C++14
입력	LiDAR 1개	LiDAR + Camera + YOLO 3개 (동기화 필요)
동기화	불필요	<code>ApproximateTime</code> slop 50 ms — 실패 시 완전 침묵
파라미터 변경	yaml 재시작	재컴파일
다운샘플	voxel 0.1 m	없음
노이즈 제거	DROR	없음
지면 제거	grid(무동작) + RANSAC(동작)	RANSAC(무동작, th=0)
관심 영역 결정	기하학적 ROI 박스	YOLO bbox (의미론적)
클러스터링	2D 거리가중 DBSCAN 변형	3D PCL Euclidean (bbox 별로 독립 수행)
크기 게이트	$0.5 \times 0.5 \times 0.3 \sim 2.5 \times 2.5 \times 1.0$ (차량급)	$0.1 \times 0.1 \times 0.1 \sim 10 \times 10 \times 10$ (사실상 노이즈 컷)
추적	Kalman → 출력에 반영됨 (coasting 有)	Kalman → 출력에 반영 안 됨 (죽은 코드)
출력 stamp	<code>rospy.Time.now()</code> ⚠	LiDAR 원본 stamp ✓
클래스 정보	없음 (기하학만)	있지만 사용 안 함
검출 가능 거리	ROI 0~8 m	ROI 0~15 m 이나 크기 게이트 때문에 실질 ~10 m 이상
검출 대상	차량급 물체만	YOLO 가 잡는 것 (현재 ERP-42 only)
강점	카메라 없이 동작. 미지 물체도 검출	오검출 적음(YOLO 가 사전 필터). 원거리 유리
약점	벽/연석/수풀 오검출. 라바콘 못 잡음	3토픽 중 하나만 끊겨도 정지. YOLO 못 잡으면 못 봄

상호 보완 구조로 설계된 것이 명확합니다. planning 쪽에서 두 소스를 별도 버퍼 (`jagyeong_obs_` , `minjae_obs_`)로 관리하고 `require_*` 플래그로 각각 필수 여부를 켜고 끌 수 있게 되어 있습니다.

5. 죽은 코드 / 버그 총정리

5.1 안전하게 삭제 가능한 것

#	패키지	대상	근거
1	detect_msgs	msg/detected_array.msg , msg/detected_object.msg + CMake 등록	리포 전체에서 참조 0
2	detect_msgs	CMake 의 roscpp , sensor_msgs	미사용
3	detect_msgs	planning_ws/src/detect_msgs 전체	중복 정의(md5 불일치 위험), planning 코드가 안 씀
4	livox_camera_fusion	cfg/LidarClustering.cfg , CMake generate_dynamic_reconfigure_options , \${PROJECT_NAME}_gencfg , dynamic_reconfigure 의존성	어디서도 include/사용 안 함. PACKAGE="Object_detect" 라는 타 프로젝트 존재
5	livox_camera_fusion	KalmanTracker , match_and_update_trackers , track_and_visualize , draw_bbox_debug , prev_centroids	출력에 영향 0 (또는 트래킹을 살릴 거면 5.3-② 참고)
6	livox_camera_fusion	read_projection_matrix 내장 기본 행렬	yaml 과 값 동일 → 이중 관리
7	livox_camera_fusion	CMake rospy	C++ 전용
8	livox_camera_fusion	"dummy" 채널	소비자 없음
9	livox_clustering	Tracker.id , KalmanFilter(q, r) 인자	미사용 / 항상 기본값
10	livox_clustering	_parse_points 래퍼	1줄 간접층

5.2 살아있지만 무의미하게 동작하는 것 (★ 실제 성능에 영향)

#	위치	증상
★1	fusion GROUND_THRESH = 0.0	RANSAC 지면 제거 완전 무효 . 30회 반복이 순수 낭비
★2	fusion ROI_RADIUS_PX=10 + CLUSTER_MIN_WIDTH/HEIGHT=0.1	~10 m 이내 물체가 크기 게이트에서 전부 탈락 할 수 있음
★3	clustering grid_min_points=10 + grid_cell_size=0.2 + voxel_size=0.1	grid 지면 제거가 지면에는 안 걸리고 수직 구조물 밑동만 깎음
4	clustering dror_max_radius=0.2	DROR 의 "동적" 특성이 rng≥1 m 에서 소멸
5	fusion BBOX_SCALE_RATIO , MIN_BBOX_EDGE_PX	ROI 원이 지배해서 효과 없음
6	fusion CLUSTER_MAX_* = 10.0	사실상 비활성

5.3 실제 버그

#	위치	내용	영향
①	양쪽 no_update() / miss()	이중 예측 — 미매칭 트래커 상태가 프레임당 2회 전진	coasting 속도 2배, 유령 위치가 실제보다 멀리 튜
②	fusion line 187-189	트래커 결과가 아니라 생 관측값을 발행	스무딩/coasting 전부 미적용
③	fusion line 367	trackers[next_tracker_id] = KalmanTracker(..., next_tracker_id++) 평가 순서 미정	지금은 무해, 향후 ID 발행 시 문제
④	clustering line 259, 264	rospy.Time.now() 로 stamp 생성	다운스트림이 LiDAR 시각과 정렬 불가

#	위치	내용	영향
⑤	clustering dist_euclid_labels line 203	non-core 점을 visited=True 로 영구 폐기	경계 점 손실, 스캔 순서 의존적 결과
⑥	fusion extract_roi 등	PCL cloud width/height 미갱신	현재는 무해, 리팩터링 시 지뢰
⑦	clustering 전역 상수 vs yaml	roslaunch 시 완전히 다른 파라미터	재현 불가능한 디버깅
⑧	detect_msgs 중복	int32 vs int64 md5 불일치	워크스페이스 섞이면 연결 거부

6. 우선순위별 개선 제안

아래는 코드 분석에 근거한 제안이며, **적용 전 실제 bag 으로 반드시 검증**하세요. 특히 P0-1, P0-2 는 현재 동작을 크게 바꿉니다.

P0 — 동작에 직접 영향

1. fusion 의 ROI 반경 & 크기 게이트 (3.4③⑥)

```
// 방법 A: 상수만 조정 (최소 수정)
static constexpr double ROI_RADIUS_PX = 40.0;    // 10 → 40
static constexpr double CLUSTER_MIN_LENGTH = 0.05;
static constexpr double CLUSTER_MIN_WIDTH  = 0.05;
static constexpr double CLUSTER_MIN_HEIGHT = 0.05;
static constexpr int    CLUSTER_MAX_SIZE   = 5000;    // 반경 키웠으니 상한도

// 방법 B: bbox 크기에 비례 (권장)
// extract_roi 에 radius 인자를 추가하고
// double r = 0.25 * std::min(box.x2-box.x1, box.y2-box.y1);
```

2. fusion 의 지면 제거 활성화 (3.4④)

```
static constexpr double GROUND_THRESH = 0.15;    // 0.0 → 0.15
```

단, ROI 반경을 키운 뒤에 적용해야 의미가 있습니다(점이 10개 미만이면 건너뛰므로).

3. livox_clustering 의 grid 지면 제거 (2.3⑥-1)

```
grid_cell_size: 0.4    # 0.2 → 0.4 (voxel 0.1 기준 16컬럼)
grid_min_points: 6     # 10 → 6
```

또는 grid 단계를 아예 제거하고 RANSAC 하나로 통일.

P1 — 정확도/안정성

4. 이중 예측 버그 (5.3①)

```
# livox_euclidean_clustering.py
def no_update(self):
    self.miss += 1    # self.kf.predict() 삭제
```

```
// livox_camera_fusion.cpp
void KalmanTracker::miss() { ++miss_count; }    // predict() 삭제
```

5. fusion 의 추적 결과를 실제로 발행 (5.3②) — 트래커를 살릴 경우

```
match_and_update_trackers(c2f, MATCH_DIST, TRACKER_MAX_MISS);
std::vector<cv::Point2d> out;
for (const auto &kv : trackers) out.emplace_back(kv.second.last_pos.x, kv.second.last_pos.y);
publish_2D_pointcloud(out, header);
```

살리지 않을 거면 5.1-⑤대로 통째로 삭제하는 게 낫습니다. **둘 중 하나로 정리**해야 합니다.

6. livox_clustering stamp (5.3④)

```
def pc_callback(self, msg):
    self.stamp = msg.header.stamp      # 저장해두고
    # publish_* 에서 rospy.Time.now() 대신 self.stamp 사용
```

7. KF 노이즈 재조정 (2.5-G)

```
class Tracker:
    def __init__(self, c, tid, dt=0.1):
        self.kf = KalmanFilter(dt, q=0.2, r=0.1)  # q 5.0 → 0.2
```

P2 — 유지보수성

8. `detect_msgs` 를 `perception_ws` 하나로 단일화 (5.1-③)
9. `cfg/LidarClustering.cfg` 및 `dynamic_reconfigure` 의존성 삭제 (5.1-④)
10. fusion 의 알고리즘 상수를 `rosparam` 또는 `yaml` 로 이관 (재컴파일 없이 튜닝)
11. `LIDAR_PITCH_DEG` 를 두 패키지가 공유하도록 (현재 0.73 이 두 곳에 중복)
12. `livox_clustering` 의 모듈 전역 기본값을 `yaml` 과 동기화 (5.3⑦)
13. `filtered_cloud` 를 비어 있어도 발행 (RViz 잔상 제거)

7. 디버깅 cheatsheet

노드가 살아있는지

```
rostopic list # /livox_euclidean_clustering, /livox_camera_fusion
rostopic hz /perception/livox/centroids
rostopic hz /perception/fusion/centroids
```

fusion 이 조용할 때 — 원인 좁히기 (위에서부터 순서대로)

```
# 1) 입력 3개가 다 살아있나
rostopic hz /livox/lidar /camera/image_raw/compressed /perception/camera/yolo

# 2) stamp 가 50 ms 안에 모이나 (ApproximateTime 실패가 1순위 원인)
rostopic echo /livox/lidar/header/stamp
rostopic echo /perception/camera/yolo/header/stamp

# 3) YOLO 가 bbox 를 내고 있나
rostopic echo /perception/camera/yolo/yolo_objects

# 4) 콜백까지 왔는데 게이트에서 죽는 건지 (디버그 클라우드가 나오면 ⑤까지는 통과)
rostopic hz /perception/fusion/filtered_cloud
```

4번에서 `filtered_cloud` 가 안 나오면 ①~⑥ 게이트 중 하나에서 전부 탈락하는 것이고, **3.4③(ROI 10px) / 3.4⑥(크기 게이트)** 가 가장 유력합니다. 확인하려면 `convert_msg` 각 `continue` 앞에 `ROS_INFO_THROTTLE` 를 임시로 넣어 어느 단계에서 몇 개가 죽는지 세어보면 즉시 나옵니다.

```
ROS_INFO_THROTTLE(1.0, "[gate] px=%zu roi=%zu ng=%zu ext=(%.3f,%.3f,%.3f)",
                  matched_px.size(), roi->size(), roi_ng->size(),
                  ext[0], ext[1], ext[2]);
```

livox_clustering 전처리 확인

```
roslaunch rviz rviz # /perception/livox/preprocessed (PointCloud2) 추가
# frame: livox_frame
rostopic echo /perception/livox/centroids/points
```

- 지면이 남아 있으면 → `ground_thresh` / grid 파라미터 (2.5-D)
- 물체가 여러 점으로 쪼개지면 → `cluster_merge_gap`, `euclidean_base_dist` (2.5-E)
- 아무것도 안 나오면 → `min_length` / `min_width` / `min_height` (2.5-F)

처리 시간

`pc_callback` 끝의 `rospy.logdebug(f"callback {...}s")` 는 기본 로그 레벨에서 안 보입니다. DEBUG 로 올려야 합니다.

```
rosservice call /livox_euclidean_clustering/set_logger_level "logger: 'rosout'
level: 'DEBUG'"
# 또는
roslaunch rqt_logger_level rqt_logger_level
# → "callback 0.xxx s" 로그 확인
```

0.1 s를 넘으면 프레임 드롭이 발생합니다. `voxel_size` 를 올리거나 ROI 를 줄이세요.

캘리브레이션(투영) 확인

`draw_bbox_debug` 가 그린 이미지를 publish 하도록 임시 수정하면 눈으로 확인 가능합니다.

```
// 헤더에 image_transport 또는 간단히
static ros::Publisher dbg = nh.advertise<sensor_msgs::Image>("debug_image", 1);
dbg.publish(cv_bridge::CvImage(header, "bgr8", camera_image).toImageMsg());
```

투영점이 bbox 중심에서 계통적으로 어긋나면 `extrinsic_matrix` 재캘리브레이션이 필요합니다.

부록: 한눈에 보는 상수 위치

값	파일	변경 방법
livox_clustering 알고리즘 전부	src/livox_clustering/config/livox_clustering.yaml	yaml 수정 → 노드 재시작
livox_clustering 코드 기본값	livox_euclidean_clustering.py:17-43	yaml 없을 때만 사용
livox_clustering KF q/r/dt	livox_euclidean_clustering.py:88	코드 수정
fusion 알고리즘 전부	include/livox_camera_fusion.h:33-65	재컴파일 필요
fusion KF 노이즈	livox_camera_fusion.cpp:27-29	재컴파일 필요
fusion 동기화 slop	livox_camera_fusion.cpp:80-81	재컴파일 필요
카메라 K / extrinsic T	config/projection.yaml (+ cpp:473-482 중복)	yaml 수정 → 재시작
YOLO 클래스 필터	src/yolov12/scripts/yolo_detect.py:46	코드 수정
토픽 이름	각 패키지 launch/*.launch	launch arg